에소테릭 언어 모델

Subham Sekhar Sahoo*[1] Zhihan Yang*[2] Yash Akhauri†[1] Johnna Liu†[1] Deepansha Singh†[1] Zhoujun Cheng†[3] Zhengzhong Liu[3] Eric Xing[3] John Thickstun[2] Arash Vahdat[4] *공동 제1 저자 †공동 제2 저자 [1] 코넬 테크 [2] 코넬 대학교 [3]MBZUAI [4] NVIDIA {ssahoo, zhihany}@cs.cornell.edu

초록

확산 기반 언어 모델은 병렬적이고 제어 가능한 생성을 가능하게 함으로써 자기회귀(AR) 모델에 대한 매력적인 대안을 제시합니다. 이 모델 계열 중 마스크드 확산 모델(MDM)은 가장 강력한 성능을 달성하지만, 퍼플렉시티 면에서 여전히 AR 모델보다 떨어지며, 특히 KV 캐싱과 같은 핵심 추론 시간 효율성 기능이 부족합니다. 본 연구에서는 AR 및 MDM 패러다임을 융합하여 각 패러다임의 퍼플렉시티 사이를 부드럽게 보간하면서 각각의 한계를 극복하는 새로운 모델 계열인 Eso-LM을 소개합니다. Eso-LM은 표준 언어 모델링 벤치마크에서 새로운 최첨단 성능을 확립합니다. 결정적으로, 우리는 병렬 생성을 유지하면서 MDM에 KV 캐싱을 도입한 최초의 사례이며, 이는 추론 효율성을 크게 향상시킵니다. 최적화된 샘플링 스케줄과 결합된 우리의 방법은 표준 MDM보다 최대 65배, 이전의 반(semi)-자기회귀 방식보다 4배 빠른 추론을 달성합니다. 코드와 모델 체크포인트는 프로젝트 페이지에서 제공됩니다:

https://s-sahoo.com/Eso-LMs

1 서론

우리는 언어 모델링의 중요한 전환점에 서 있습니다. 패러다임 전환이 진행 중입니다. 오랫동안 황금 표준으로 여겨졌던 자기회귀(AR) 언어 모델이 이제 표준 언어 생성에서 확산 언어 모델과 경쟁하고 있습니다. 최근 연구 [26, 29]는 마스크드 확산 모델(MDM)이 소규모 언어 벤치마크에서 AR 모델과의 격차를 좁히고 있으며, 분자 생성 [28, 11] 및 그래프 생성

이러한 결과는 MDM을 AR 모델에 대한 매력적인 대안으로 만듭니다. 하지만 MDM은 두 가지 주요 한계를 가지고 있습니다: (1) 추론 속도: 병렬 생성을 지원함에도 불구하고, MDM은 실제로는 AR 모델보다 상당히 느립니다. 이는 주로 채팅 시스템과 같은 실시간 애플리케이션에 필수적인 최적화인 KV 캐싱이 부족하기 때문입니다. (2) 생성 품질: MDM은 더 복잡한 언어 모델링 작업에서 여전히 현저한 가능성(likelihood) 격차를 보입니다

최근 제안된 BD3-LM

---

img-0.jpeg Figure 1: Efficient generation of an example sequence with our flagship model Eso-LM (B). During Diffusion Phase, Eso-LMs denoise one or more, potentially non- neighboring mask tokens (M) per step. During Sequential Phase, Eso-LMs denoise the remaining mask tokens one at a time from left to right. Eso-LM (B) allows for KV caching in both phases using just a single unified KV cache: blue bounding boxes enclose transformer cells that are building their KV cache; a cell becomes blue once its KV cache is built. The sequences below the transformers depict tokens in their natural order.

캐싱: 블록 간 KV 캐싱은 가능하지만, 블록 내 확산은 여전히 KV 지원이 부족하여 전반적인 속도 향상을 제한합니다.

이러한 과제를 해결하기 위해 우리는 자기회귀 및 마스크드 확산 접근 방식을 융합한 새로운 언어 모델링 패러다임을 제안합니다. 우리 모델은 AR 및 MDM 목표의 조합인 하이브리드 손실로 학습되며, 이를 통해 두 패러다임의 퍼플렉시티

사이를 부드럽게 보간할 수 있습니다. 이를 위해서는 두 가지 핵심 혁신이 필요합니다: (1) AR 및 MDM 생성 방식을 모두 지원하기 위한 노이즈 제거 트랜스포머의 수정된 어텐션 메커니즘. (2) 확산 단계 내에서 KV 캐싱을 가능하게 하는 새로운 훈련 및 샘플링 절차. 이는 이전 MDM에서는 사용할 수 없었던 기능입니다. 이러한 하이브리드 설계의 비전통적인 특성 때문에 우리는 우리의 방법을 에소테릭 언어 모델(Eso-LM)이라고 명명합니다.

우리의 방법은 언어 모델링 벤치마크에서 확산 모델 중 새로운 최첨단 성능을 확립하고, AR 및 MDM 퍼플렉시티 간의 보간에 대한 세밀한 제어를 가능하게 합니다. 이전 접근 방식과 달리, 우리의 방법은 확산 중 KV 캐싱을 지원하여 훨씬 빠른 추론을 가능하게 합니다. 요약하면, 우리의 기여는 다음과 같습니다: (1) 우리는 AR 및 MDM 패러다임을 융합하고 이전의 하이브리드 접근 방식인 BD3-LM을 능가하는 새로운 언어 모델링 프레임워크를 제안합니다. (2) 우리는 병렬 생성을 유지하면서 MDM에 대한 KV 캐싱을 가능하게 한 최초의 접근 방식이며, 이는 표준 MDM보다 최대 65×배, KV 캐싱이 적용된 반(semi)-자기회귀 기준선보다 3–4×배 더 빠른 추론을 달성합니다 (섹션 5.2). (3) 우리는 BD3-LM이 낮은 샘플링 단계에서 성능이 저하되는 반면, 우리의 방법은 낮은 NFE 체제에서는 MDM과, 높은 NFE 체제에서는 AR과 경쟁력이 있음을 보여줍니다 (섹션 5.3). (4) 우리의 결과는 이산 확산에 대한 새로운 최첨단 퍼플렉시티를 확립하여 자기회귀 모델과의 격차를 좁힙니다 (섹션 5.1).

2 배경

표기법 우리는 K개의 값을 가질 수 있는 스칼라 이산 확률 변수를 '원-핫' 열 벡터로 표현하고, $\mathscr{V}\in\mathbf{x}\in0,1K:\Sigma i=1K\mathbf{x}i=1$을 그러한 모든 벡터의 집합으로 정의합니다. $\mathbf{m}\in\mathscr{V}$를 $\mathbf{m}K=1$인 특별한 마스크 벡터라고 합시다. $Cat(\cdot;\pi)$를 K개의 클래스에 대한 범주형 분포로 정의하며, 확률은 $\pi\in\Delta K$에 의해 주어지고, 여기서 ΔK는 K 심플렉스를 나타냅니다. 또한, $\langle\mathbf{a},\mathbf{b}\rangle$를 벡터 $\mathbf{a}$와 $\mathbf{b}$ 사이의 내적으로 나타냅니다. 우리는 순서 있는 집합(튜플)을 나타내기 위해 괄호 ()를 사용하고, 순서 없는 집합을 나타내기 위해 중괄호 를 사용합니다. |A|는 집합 A의 원소 개수를 나타냅니다.

MDM은 두 가지 주요 순서(orderings)를 특징으로 합니다: 시퀀스 순서와 노이즈 제거 순서. 우리는 이들 순서 간의 관계를 설명하기 위해 순열 σ을 사용합니다. $\mathscr{P}L$을 [L]=1,⋯,L의 모든 순열의 집합으로 나타냅시다. $\sigma\in\mathscr{P}L$은 순서 있는 집합(튜플)이며 전단사 함수로도 기능합니다: σ(l)는 σ의 lth 위치에 있는 항목이고, σ−1(i)는 σ에서 항목 i의 위치입니다. 예를 들어, (2,4,1,3)은 1,2,3,4의 순서를 나타내며, σ(2)=4는 위치 2를 해당 항목 4에 매핑하고, σ−1(4)=2는 항목 4를 해당 위치 2에 매핑합니다.

$\mathbf{x}\in\mathscr{V}L$을 마스크 토큰이 없는 길이 L의 시퀀스로 나타내고, $\mathbf{x}\ell$를 $\mathbf{x}$의 ℓth번째 항목으로 나타냅시다. 우리는 '토큰 인덱스'라는 용어를 원래 순서에서 토큰의 위치를 지칭하는 데 사용합니다. 예를 들어, $\mathbf{x}$l의 토큰 인덱스는 l입니다. (z<i>t)</i>t∈[0,1]∈$\mathscr{V}$L을 마스크 토큰을 포함할 수 있는 길이 L의 시퀀스로 나타냅시다. $\mathscr{M}(\mathbf{z}t)=\ell|\mathbf{z}t\ell=\mathbf{m}$를 zt의 마스크 토큰의 토큰 인덱스로 나타내고, $\mathscr{C}(\mathbf{z}t)=\ell|\mathbf{z}t\ell\neq\mathbf{m}$을 zt의 클린 토큰의 토큰 인덱스로 나타냅시다.

⊕:$\mathscr{V}$m×$\mathscr{V}$n→$\mathscr{V}$m+n을 길이 m과 n인 두 시퀀스 $\mathbf{x}=(\mathbf{x}1,\mathbf{x}2,\cdots,\mathbf{x}m)$와 $\mathbf{z}=(\mathbf{z}1,\mathbf{z}2,\cdots,\mathbf{z}n)$에 대한 연결(concatenation) 연산자로 나타냅시다. 연결된 시퀀스 $\mathbf{x}\oplus\mathbf{z}$가 트랜스포머에 입력될 때, $\mathbf{x}$와 $\mathbf{z}$은 각각 독립적으로 트랜스포머에 입력될 때와 동일한 위치 임베딩을 가집니다. ⊙:$\mathscr{V}$m×$\mathscr{V}$n→$\mathscr{V}$m을 대체(substitution) 연산자로 나타냅시다; m>n인 임의의 $\mathbf{z}\in\mathscr{V}$m와 $\mathbf{x}\in\mathscr{V}$n에 대해, 출력 $\mathbf{y}=\mathbf{z}\odot\mathbf{x}$는 다음으로 주어집니다: $\mathbf{y}1:n=\mathbf{x}$ 및 $\mathbf{y}n+1:m=\mathbf{z}n+1:m$.

2.1 자기회귀 모델

$\mathscr{V}$L 내의 시퀀스 $\mathbf{x}$~qdata가 주어지면, 자기회귀(AR) 모델은 결합 분포의 다음 인수분해를 정의합니다: $\log p\theta(\mathbf{x})=\Sigma t=1L\log p\theta(\mathbf{x}\ell|\mathbf{x}c\ell)$, 여기서 모델 pθ은 일반적으로 인과적 트랜스포머

2.2 마스크드 확산 모델

확산 모델은 순방향 손상(corruption) 프로세스 q를 역변환하는 방법을 학습하며, 이 프로세스는 $\mathscr{V}^L$ 내의 클린 데이터 $\mathbf{x}\sim q_{\text{data}}$를 $t\in[0,1]$에 대한 잠재 변수 시퀀스 $\mathbf{z}_t$으로 변환하고, 각 잠재 변수는 점점 더 노이즈가 많은 $\mathbf{x}$ 버전을 나타냅니다 [31, 32, 8]. MDM [26, 30, 20]에서 순방향 마스킹 프로세스 q는 시퀀스 $\mathbf{x}$에 걸쳐 독립적으로 인수분해됩니다. 즉, $q_t(\cdot|\mathbf{x})=\prod_\ell q_t(\cdot|\mathbf{x}^\ell)$이고 각 토큰 $\mathbf{x}^\ell$는 고정된 목표 분포 $\text{Cat}(\cdot;\mathbf{m})$와 점진적으로 보간됩니다. 시간 t에서의 $\mathbf{z}_t^\ell\sim q_t(\cdot|\mathbf{x}^\ell)$의 주변은 다음과 같이 주어집니다:

$$q_t(\cdot|\mathbf{x}^\ell)=\text{Cat}(\cdot;\alpha_t\mathbf{x}^\ell+(1-\alpha_t)\mathbf{m}), \quad (1)$$

여기서 $\alpha_t\in[0,1]$는 $\alpha_0\approx 1$와 $\alpha_1\approx 0$을 갖는 t의 엄격히 감소하는 함수입니다. Sahoo 등

$$[\, q_{s|t}(\cdot|\mathbf{z}_t^\ell,\mathbf{x}^\ell)=\begin{cases}\text{Cat}(\cdot;\mathbf{z}_t^\ell) & \mathbf{z}_t^\ell\neq\mathbf{m},\\ \text{Cat}\left(\cdot;\frac{(1-\alpha_s)\mathbf{m}+(\alpha_s-\alpha_t)\mathbf{x}^\ell}{1-\alpha_t}\right) & \mathbf{z}_t^\ell=\mathbf{m}.\end{cases} \,] \quad (2)$$

노이즈 제거 모델 $\mathbf{x}_\theta:\mathscr{V}^L\to(\Delta^K)^L$가 주어지면, 시퀀스 $\mathbf{z}_s$에 대한 역 마스크 해제 프로세스 $p_\theta(\cdot|\mathbf{z}_t)_{s|t}$는 다음으로 매개변수화됩니다.

$$p_\theta(\cdot|\mathbf{z}_t)_{s|t}=\prod_\ell p_\theta(\cdot|\mathbf{z}_t)_{s|t}=\prod_\ell q_{s|t}(\cdot|\mathbf{z}_t^\ell,\mathbf{x}^\ell=\mathbf{x}_\theta^\ell(\mathbf{z}_t)). \quad (3)$$

Sahoo 등

$$\mathscr{L}_{\text{MDM}}(\mathbf{x})=\mathbb{E}_{q,t\sim[0,1]}\alpha'_t\sum_{\ell\in\mathscr{M}(\mathbf{z}_t)}\log\langle\mathbf{x}_\theta^\ell(\mathbf{z}_t),\mathbf{x}^\ell\rangle, \quad (4)$$

이는 마스크된 토큰 $\mathscr{M}(\mathbf{z}_t)$에 대해서만 계산된 마스크드 언어 모델링 손실

길이 L의 시퀀스를 생성하기 위해, 역 확산 프로세스는 $\mathbf{z}_{t=1}$에서 시작하며, 여기서 $\ell=1,\cdots,L$에 대해 $\mathbf{z}_{t=1}^\ell=\mathbf{m}$입니다. 이 프로세스는 T 단계에 걸쳐 진행되며, 각 $\mathbf{z}_s^\ell$는 (3)에 정의된 대로 $p_\theta(\cdot|\mathbf{z}_t)_{s|t}$에서 독립적으로 샘플링됩니다; 일단 위치가 마스크 해제되면, 그 위치는 고정됩니다. 여러 토큰이 병렬로 노이즈 제거될 수 있으므로, 총 네트워크 함수 평가(NFE) 수는 L보다 작을 수 있어 더 빠른 생성을 가능하게 합니다. 그러나 $\mathbf{x}_\theta(\mathbf{z}_t)$ 내의 양방향 트랜스포머로 인해 각 NFE는 비용이 많이 듭니다.

2.3 블록 노이즈 제거 확산 이산 언어 모델

블록 노이즈 제거 확산 이산 언어 모델(BD3-LM)

블록 크기에서, BD3-LM은 AR 모델과 MDM을 보간한다. BD3-LM은 $\mathbf{x}$의 토큰을 $B=L/L'$을 사용하여 L'개의 연속적인 토큰으로 구성된 B개의 블록으로 그룹화하며, 여기서 B는 정수라고 가정한다. $\mathbf{x}$에 대한 우도는 블록에 대해 $-\log p_\theta(\mathbf{x})=-\sum_{b=1}^B\log p_\theta(\mathbf{x}^b|\mathbf{x}^{<b})\le\sum_{b=1}^B\mathscr{L}_{\text{MDM}}(\mathbf{x}^b;\mathbf{x}^{<b})$과 같이 자기회귀적으로 분해되며, 여기서 p_θ

i>θ(xb|x<b)은 조건부 MDM이고 LMDM(xb;x<b)은 (4)에 정의된 MDLM에 대한 NELBO로, 블록에 걸쳐 순차적으로 적용된다.

3 에소테릭 언어 모델

우리는 언어 모델링을 위한 새로운 패러다임인 에소테릭 언어 모델(Eso-LM)을 제안하며, 이는 AR 모델과 MDM의 공생적 조합을 이룬다. 우리의 접근 방식은 두 가지 주요 구성 요소를 갖는다: (1) 순차적(AR 방식) 및 병렬(MDM 방식) 생성 모두를 지원하는 모델군을 소개하고, (2) 4절에서는 이러한 하이브리드 생성을 가능하게 하는 트랜스포머

AR 모델은 현재 언어 모델링 성능에서 앞서지만, 토큰을 순차적으로 생성하여 추론 속도가 느리다. 이와 대조적으로, MDM은 여러 토큰을 병렬로 생성하며 더 제어 가능하지만 [29, 19], 일반적으로 AR 모델보다 더 높은 혼란도(perplexity)를 보인다 [26, 27]. 핵심 질문은: 이들의 강점을 결합할 수 있는가? 우리는 일부 토큰은 MDM을 통해 병렬로 생성하고 나머지는 왼쪽에서 오른쪽으로 순차적으로 생성하는 하이브리드 접근 방식을 소개한다. 이는 두 가지 핵심 질문을 제기한다. 첫째, 이러한 생성 모델의 우도(likelihood)를 계산할 수 있는가? 우리는 Eso-LM이 참 우도에 대한 원칙적인 경계(bound)를 허용한다는 것을 보여준다. 둘째, 트랜스포머의 어텐션 메커니즘을 두 가지 생성 방식을 모두 지원하도록 어떻게 조정할 수 있는가? 훈련 중에 AR 모델에서는 클린 시퀀스에 인과적 어텐션이 적용되는 반면, MDM은 마스킹된 토큰과 마스킹되지 않은 토큰 모두에 대한 양방향 어텐션에 의존한다. 4절에서는 트랜스포머를 위한 맞춤형 어텐션 메커니즘을 설계하여 이 두 가지 상이한 아키텍처를 조화시킨다. 또한, 어텐션 행렬에 희소성을 도입하여 MDM을 위한 KV 캐싱을 가능하게 하며, 이는 긴 컨텍스트 생성 속도를 크게 향상시킨다.

3.1 자기회귀 모델과 마스킹된 확산 융합

VL에서 x~qdata(x)을 데이터 분포에서 추출한 샘플이라고 하고, θ으로 매개변수화된 우리 모델 분포를 pθ라고 하자. Eso-LM은 pθ를 두 가지 구성 요소, 즉 AR 모델 pθAR와 MDM pθMDM으로 분해한다. MDM은 부분적으로 마스킹된 시퀀스 z0∈VL~pθMDM(z0)을 생성하고, AR 모델은 나머지 마스킹 해제 단계를 pθAR(x|z0)과 같이 자기회귀적인 왼쪽-오른쪽 방식으로 완료한다. 이러한 하이브리드 생성 프로세스의 주변 우도(marginal likelihood)는 다음과 같다.

pθ(x)=Σz0∈VLpθAR(x|z0)pθMDM(z0). (5)

이 합은 다루기 어렵지만(intractable), 후방 분포(posterior) q(z0|x)

−logpθ(x) ≤−E<i>z</i>0~q0(.|x)[logpθAR(x|z<i>0)]+D</i>KL(q0(z<i>0|x)|p</i>θMDM(z<i>0)) (6) =−E</i>z</i>0~q</i>0(.|x)[Σℓ∈M(z<i>0)logp</i>θAR(xℓ|z<i>0,x<ℓ)]+D</i>KL(q0(z<i>0|x)|p</i>θMDM(z0)).

여기서 pθAR(x|z0)는 z0의 클린 토큰에 조건화된, 마스킹된 위치 ℓ∈M(z0)에 대한 결합 AR 우도이다. AR의 경우, 디노이징 네트워크 xθ:VL→(ΔK)L은 입력 z0⊙x<ℓ에서 작동하며, 여기서 대체 연산자 ⊙는 z0의 첫 l−1개 토큰을 x<ℓ으로 대체한다. 각 ℓ∈M(z0)에 대해, xθℓ(z0⊙x<ℓ)은 x<ℓ와 z0이 주어졌을 때 클린 토큰 xℓ의 분포를 근사하며, 이는 위치 ℓ를 넘어선 클린 토큰을 포함할 수 있다. 부록 B.1에서, 우리는 KL 항을 분석하고 NELBO가 다음임을 보여준다.

LNELBO(x)=E<i>z</i>0~q</i>0[−Σℓ∈M(z<i>0)log⌈x</i>θℓ(z<i>0⊙x<ℓ),xℓ⌋]⎵</i>AR loss+∫t=0t=1α′1−αtE<i>z</i>t~qt[Σℓ∈M(z<i>t)log⌈x</i>θℓ(z<i>t),xℓ⌋]⎵</i>MDM lossdt. (7)

AR과 확산 사이 보간

α0=1일 때, 후방 분포 샘플 $\mathbf{z}^{i>0}=\mathbf{x}$과 모든 토큰은 MDM에 의해 생성된다. 따라서 (7)에서 AR 손실은 0이 되고, $\mathscr{L}^{i>}$NELBO는 MDM 손실로 축소된다. 반대로, α0=0일 때, 모든 토큰이 마스킹되고 MDM 손실은 사라지며, $\mathscr{L}^{i>}$NELBO는 AR 손실로 축소된다. 따라서 Eso-LM은 하이퍼파라미터 $\alpha^{i>0}$에 의해 제어되는 AR 및 MDM 패러다임 사이를 보간한다.

3.2 샘플링

우리는 (5)에 정의된 2단계 샘플링을 수행한다: $p_\theta(\mathbf{x})$에서 샘플링하기 위해, 먼저 부분적으로 마스킹된 시퀀스 $\mathbf{z}^{i>0}\sim p^{i>}\theta$MDM을 뽑고, 그 다음 pθAR를 사용하여 이를 완성한다.

확산 단계

완전히 마스킹된 시퀀스 $\mathbf{z}^{i>}t=1=\mathbf{m}1{:}L$에서 시작하여, 표준 조상 샘플링(2.2절)은 각 단계에서 마스킹된 토큰의 부분집합만 업데이트하면서 전체 시퀀스에 대한 정방향 전달(forward pass)을 수행하여 FLOP를 낭비한다. 효율성을 개선하기 위해, 우리는 표준 샘플링 및 훈련 절차에 두 가지 주요 변경 사항을 제안한다. 샘플링 중, 우리는 확산 디노이징 스케줄 $\mathcal{S}$MDM=(S$^{i>}$1,⋯,S1/T)을 사전 계산하며, 여기서 St는 확산 단계 t에서 디노이징될 마스크 토큰 인덱스의 집합이다. 전체 시퀀스를 처리하는 대신, 우리는 부분 시퀀스 $\mathbf{z}^{i>}t\ell|\ell\in\mathcal{G}(\mathbf{z}^{i>}t)\cup$St에 대해서만 정방향 전달을 수행한다. 즉, 클린 토큰과 디노이징이 예정된 토큰들만 처리하여, 특히 긴 시퀀스에서 계산량을 크게 줄인다. 이러한 샘플링 방식을 맞추기 위해, 모델은 훈련 시 마스크 토큰에 대한 양방향 어텐션을 피해야 한다 (4절에서 논의된 바와 같이). 이 접근 방식은 또한 확산 중 효율적인 KV 캐싱을 가능하게 한다 (4.2절에서 논의된 바와 같이).

순차 단계

우리는 최종 출력 $\mathbf{x}\sim$을 $\mathbf{z}^{i>}0$로 초기화한다. 그 다음 AR 모델은 순차적 디노이징 스케줄 $\mathcal{S}$AR=((i)|i∈$\mathcal{M}(\mathbf{z}^{i>}0)$)를 사용하여 마스크 토큰을 왼쪽에서 오른쪽으로 채우는데, 이 스케줄에 대해 우리는 단일 요소 집합이 그 유일한 요소에 의해 오름차순으로 정렬되도록 요구한다. 표준 AR 디코딩과 달리, 각 $\mathbf{x}\sim\ell$은 완전히 클린 토큰으로 구성된 왼쪽 컨텍스트와 $\mathbf{z}0$의 오른쪽 클린 토큰 모두에 조건화되어 더 풍부한 생성을 가능하게 한다. 우리는 오른쪽 마스크 토큰에 대한 평가를 건너뛰어 불필요한 계산을 줄인다. 순차 단계는 자연스럽게 KV 캐싱을 지원한다.

우리는 통합 디노이징 스케줄을 $\mathcal{S}=\mathcal{S}$MDM∪$\mathcal{S}$AR으로 나타내며, 이는 [L]을 분할하기 위해 두 샘플링 스케줄을 연결한 것이다. α0=1일 때, 모든 토큰은 확산에 의해 생성되므로, $\mathcal{S}=\mathcal{S}$MDM 및 $\mathcal{S}$AR=∅ (널 집합)이다. α0=0일 때, 모든 토큰은 순차적으로 생성되므로, $\mathcal{S}=\mathcal{S}$AR 및 $\mathcal{S}$MDM=∅이다. 전체 샘플링 알고리즘 및 예시 그림은 부록 B.4를 참조하라.

4 공유 디노이징 트랜스포머를 위한 어텐션 메커니즘 설계

본 섹션에서는 공유 트랜스포머 아키텍처를 사용하여 순차적(AR) 및 병렬(MDM) 생성 모드를 모두 지원하는 통합 모델링 프레임워크를 소개한다. 우리의 핵심 기술적 기여는 인과적 어텐션과 한 칸 이동 디코딩을 요구하는 AR 모델과 양방향 어텐션에 의존하는 MDM 사이의 아키텍처 불일치를 조정하는 유연한 어텐션 메커니즘이다. 이를 달성하기 위해, 우리는 어텐션 바이어스 행렬 A∈−∞,0L′×L′을 도입하며, 여기서 L′은 시퀀스 길이이고, 이는 표준 어텐션을 다음과 같이 조절한다.

₩textscSelf−Attention(Q,K,V,A)=softmax(QK′d+A)V

여기서 Q,K,V∈ℝL′×d는 쿼리, 키, 값 행렬을 나타낸다. A의 엔트리(값)들은 정보 흐름을 제어한다: Ai,j=0는 토큰 i에서 j로의 어텐션을 "허용"하고, Ai,j=−∞은 "차단"한다. 이 메커니즘은 단일 트랜스포머가 필요에 따라 인과적 및 양방향 동작을 모두 에뮬레이트할 수 있도록 한다.

이 통합 어텐션 방식을 바탕으로, 우리는 두 가지 변형인 Eso-LM (A)와 Eso-LM (B)를 제안한다. Eso-LM (A)는 어텐션을 희소화하고 각 확산 단계에서 디노이징 트랜스포머를 마스크 토큰의 부분집합에 선택적으로 적용함으로써 계산량 감소에 중점을 둔다. Eso-LM (B)는 마스크 토큰뿐만 아니라 클린 토큰에도 인과적 마스킹을 도입하여 이 아이디어를 더욱 확장하고, 이는 약간의 혼란도 감소를 대가로 훨씬 더 큰 속도 향상을 위한 KV 캐싱을 가능하게 한다. 본 섹션의 나머지 부분에서는 이러한 통합을 가능하게 하는 어텐션 공식화와 확산 및 순차 단계에서 이 두 변형의 차이점을 자세히 설명한다.

확산 단계

표준 MDM은 샘플링 중 두 가지 비효율성을 겪는다: (i) 부분 업데이트에도 불구하고 전체 시퀀스 정방향 전달, (ii) 양방향

---

img-1.jpeg Figure 2: Comparison of attention biases for diffusion-phase training. Orange represents 0 (attention) and gray represents −∞ (no attention). The clean sequence is $\mathbf{x}$=(A,B,C,D,E,F) . After random masking, we obtain $\mathbf{z}$t=(A,M,C,M,M,F) . The integers denote the position indices with $\mathscr{M}(\mathbf{z}t)$=2,4,5 and $\mathscr{C}(\mathbf{z}t)$=1,3,6 . The random ordering is σ=(3,1,6,4,5,2)~$\mathscr{P}$6 with clean tokens before mask tokens. Red highlights differences between Eso-LM (A) and Eso-LM (B).

img-2.jpeg

img-3.jpeg

어텐션으로 인한 단계 간 KV 캐싱 부재. 3.2절에서 간략히 논의했듯이, 우리는 단계 t에서 $\mathbf{x}$R을 $\mathbf{z}$ti|i∈$\mathscr{C}$($\mathbf{z}$t)∪St에 대해서만 평가함으로써 (i)을 해결한다. 이러한 가능성을 열기 위해, 우리는 AO-ARM

순차 단계 순차 생성 중, 모델은 ($\mathbf{z}$0i)t∈$\mathscr{M}$($\mathbf{z}$0)이(가) $\mathbf{z}$0~pθMDM(.)에 의해 생성된 마스크 토큰의 노이즈를 왼쪽에서 오른쪽으로 제거해야 합니다. 이는 생성 중에는 간단하지만 훈련 중에는 어렵습니다. 각 토큰이 왼쪽만 참조하는 표준 AR 모델과 달리, 각 마스크 토큰 $\mathbf{z}$0i는 (i) 자기 자신(위치 임베딩용), (ii) 깨끗한 왼쪽 문맥 $\mathbf{x}$<i($\mathbf{z}$0<i의 모든 이전 마스크 토큰이 채워졌다고 가정), (iii) 그리고 오른쪽 문맥 $\mathbf{x}$jIj∈$\mathscr{C}$($\mathbf{z}$0),j>i 또는 등가적으로 $\mathbf{z}$0jIj∈$\mathscr{C}$($\mathbf{z}$0),j>i의 깨끗한 토큰들을 참조해야 합니다. 훈련 중에 (ii)를 처리하는 것은 간단하지 않습니다. 왜냐하면 $\mathbf{z}$0의 각 마스크 토큰은 자신의 왼쪽에 있는 토큰들이 이미 pθAR에 의해 노이즈가 제거되었다고 가정하기 때문입니다. 이러한 동작을 시뮬레이션하기 위해, 훈련 중에 연결된 시퀀스 $\mathbf{z}$0⊕$\mathbf{x}$를 트랜스포머에 입력하고 모든 마스크 토큰에 대해 올바른 정보 흐름을 강제하는 사용자 정의 어텐션 마스크 A를 구성합니다. 손실은 (7)에 따라 $\mathbf{z}$0의 마스크된 토큰에 해당하는 로짓에 대해서만 계산됩니다. 생성 중에는 이 문제가 자연스럽게 해결되며, 연결이 필요하지 않습니다.

4.1 Eso-LM (A)

우리는 Eso-LM (A)를 소개합니다. 이는 확산 단계 동안 노이즈 제거 트랜스포머에서 마스크 토큰에 대한 양방향 어텐션을 제거합니다. 이 수정은 3.2절에 설명된 샘플링 스케줄을 사용할 수 있게 하여 더 빠른 추론을 가능하게 합니다. Eso-LM (A)는 순차 샘플링 중에만 KV 캐싱을 지원하지만, 확산 샘플링 중에는 표준 MDM보다 훨씬 적은 계산을 수행합니다.

4.1.1 훈련

확산 단계 노이즈 제거 트랜스포머는 노이즈를 제거할 마스크 토큰을 포함하는 $\mathbf{z}_t \sim q_t(\cdot|\mathbf{x})$을(를) 입력으로 받고, $\mathbf{x}$을(를) 목표로 받습니다. σ에서 $\mathbf{z}_t$의 깨끗한 토큰이 $\mathbf{z}_t$의 마스크 토큰보다 먼저 오는 자연스러운 제약 조건으로 무작위 순서 $\sigma \sim \mathscr{P}_L$를 샘플링합니다. 우리는 다음으로 L×L 어텐션 편향을 정의합니다.

$$A_{i,j} = \begin{cases} 0 & \forall (i,j) \in C(z_t) \times C(z_t) \\ 0 & \text{if } \sigma^{-1}(i) \geq \sigma^{-1}(j) \forall (i,j) \in M(z_t) \times [L] \\ -\infty & \text{otherwise.} \end{cases} \tag{10}$$

깨끗한 토큰 $\mathbf{z}_{t|i} \in \mathscr{C}(\mathbf{z}_t)$은(는) 그들 사이에서 양방향 어텐션(8)을 가지며, 마스크 토큰 $(\mathbf{z}_t)_i \in \mathscr{M}(\mathbf{z}_t)$은(는) 깨끗한 토큰, 자기 자신, 그리고 σ(9)에 따른 이전 마스크 토큰을 참조합니다. 우리는 다음을 무시합니다.

σ의 깨끗한 토큰들 간의 순서를 무시하고 단순히 그들 간의 양방향 어텐션을 사용합니다. 예시는 그림 2를 참조하십시오. 흥미롭게도, A은(는) A의 행과 열을 σ으로 정렬하면 Prefix-LM

순차 단계 노이즈 제거 트랜스포머는 연결된 시퀀스 $\mathbf{z}_0 \oplus \mathbf{x} \in \mathscr{V}^{2L}$을(를) 입력으로 받는데, 여기서 $\mathbf{z}_0 \sim q_0(\cdot|\mathbf{x})$은(는) 노이즈를 제거할 마스크 토큰을 포함하고, $\mathbf{x}$은(는) 목표입니다. 우리는 2L×2L 어텐션 편향을 다음으로 정의합니다.

$A_{i,j}=0$	if $i=j \forall (i,j) \in M(z_0) \times M(z_0)$	(11)
$A_{i,j+L}=0$	$\forall (i,j) \in M(z_0) \times C(z_0)$	(12)
$A_{i,j+L}=0$	if $i>j \forall (i,j) \in M(z_0) \times M(z_0)$	(13)
$A_{i+L,j+L}=0$	$\forall (i,j) \in C(z_0) \times C(z_0)$	(14)
$A_{i+L,j+L}=0$	$\forall (i,j) \in M(z_0) \times C(z_0)$	(15)
$A_{i+L,j+L}=0$	if $i \geq j \forall (i,j) \in M(z_0) \times M(z_0)$	(16)
$A_{i,j}=-\infty$	otherwise.	(17)

노이즈가 있는 문맥 $\mathbf{z}_0$에 있는 토큰에 대한 어텐션 메커니즘은 다음과 같이 설명됩니다. 마스크 토큰 $(\mathbf{z}_{i>0})_i \in \mathscr{M}(\mathbf{z}_0)$은(는) (i) 자기 자신(11), (ii) 깨끗한 토큰 $\mathbf{x}_{j|j} \in \mathscr{C}(\mathbf{z}_0)$(12), 그리고 (iii) 왼쪽에 있는 마스크 토큰의 깨끗한 버전 $\mathbf{x}_{j|j} \in \mathscr{M}(\mathbf{z}_0), i>j$(13)에 어텐션합니다. 깨끗한 토큰 $(\mathbf{z}_{i>0})_i \in \mathscr{C}(\mathbf{z}_0)$는 다른 어떤 토큰도 참조하지 않으므로 무엇이든 참조할 수 있습니다(17). 깨끗한 문맥 $\mathbf{x}_0$에 있는 토큰에 대한 어텐션 메커니즘은 다음과 같이 설명됩니다. 토큰 $\mathbf{x}_{i|i} \in \mathscr{C}(\mathbf{z}_{i>0})$은(는) 양방향 어텐션(14)을 가집니다. 마스크 토큰에 해당하는 깨끗한 토큰인 $(\mathbf{x}_i)_i \in \mathscr{M}(\mathbf{z}_0)$은(는) $\mathbf{x}_{j|j} \in \mathscr{C}(\mathbf{z}_0)$(15)과 $\mathbf{x}_{j|j} \in \mathscr{M}(\mathbf{z}_0), i \geq j$(16)에 어텐션합니다.

예시는 그림 6을 참조하십시오. σ이(가) 두 가지 속성을 가진 순서라고 가정합니다. 즉, (i) σ에서 z0의 깨끗한 토큰이 z0의 마스크 토큰보다 선행하고, (ii) σ에서 z0의 마스크 토큰이 자연스러운 순서로 배열됩니다. 다시 말해, 깨끗한 토큰 xi∣i∈C(z0) 간의 순서는 무시되며, 그들 간에는 양방향 어텐션을 사용합니다. 네 개의 L-by-L 블록 각각의 행과 열이 σ로 정렬되면, A는 희소하고 패턴화된 블록 구조를 보여주며(그림 8), 우리는 이를 구현에 활용합니다(부록 B.5).

4.1.2 샘플링

확산 또는 순차 샘플링 동안, 부분적으로 마스킹된 시퀀스 zk이(가) 주어지면, 노이즈 제거 모델은 K=∣S∣인 Sk∈S=S1,⋯,SK에 대해 마스크 토큰 zki∣i∈Sk의 노이즈를 제거해야 합니다. 우리는 토큰의 부분 집합 zki∣i∈C(zk)∪Sk에 대해 순방향 패스를 수행합니다. 토큰의 부분 집합에 대해 순방향 패스를 수행하는 동안, 실제 시퀀스에서 이 토큰들의 위치 임베딩이 보존된다는 점에 유의하는 것이 중요합니다. 아래에서 순방향 패스에 사용된 어텐션 편향을 설명합니다.

k 단계 이전에 확산 단계에서 디코딩된 토큰들의 인덱스 집합을 DkMDM이라 하고, 순차 단계에 대한 인덱스 집합을 DkAR라 합시다. S로 정의된 토큰들을 디노이즈하는 순서를 σ이라 합시다. 우리는 k 단계에서의 L×L 어텐션 편향을 다음과 같이 정의합니다.

Ai,j=	**0**	**∀(i,j)∈DkMDM×DkMDM**	**(18)**
	0	∀(i,j)∈DkAR×DkMDM	(19)
	0	if i≥j∀(i,j)∈DkAR×DkAR	(20)
	0	∀(i,j)∈Sk×(DkMDM∪DkAR)	(21)
	0	if σ-1(i)≥σ-1(j)∀(i,j)∈Sk×Sk	(22)
	-∞	otherwise.	(23)

확산 단계에서 디코딩된 깨끗한 토큰 zki∣i∈DkMDM은(는) 서로 양방향 어텐션(18)을 가집니다. 순차적으로 디코딩된 깨끗한 토큰 (zki)i∈DkAR은(는) 확산 단계에서 디코딩된 깨끗한 토큰 zkj∣j∈DkMDM(19), 자기 자신, 그리고 이전에 순차적으로 디코딩된 깨끗한 토큰 zkj∣j∈DkAR,i>j(20)에 어텐션합니다. 노이즈를 제거할 마스크 토큰 (zki)i∈Sk는 디코딩된 모든 깨끗한 토큰 zkj∣j∈DkMDM∪DkAR(21), 자기 자신, 그리고 σ에 따른 이전 마스크 토큰들(zkj∣j∈Sk,σ−1(i)>σ−1(j)) (22)에 어텐션합니다. 노이즈 제거 스케줄에 포함되지 않은 마스크 토큰 (zki)i∈S>k은(는) 다른 토큰이 자신을 참조하지 않으므로 무엇이든 참조할 수 있습니다(23).

(i) (zki)i∈C(zk)∪Sk이(가) zki∣i∈S>k을(를) 참조하지 않고, (ii) 깨끗한 토큰 샘플링을 위해 zki∣i∈Sk에 대한 트랜스포머 출력만 사용하므로, zki∣i∈S>k에 대한 모든 계산을 피할 수 있어 확산 단계에서 계산량을 현저히 줄일 수 있습니다. 그림 9는 이 샘플링 절차가 순차 단계 중 KV 캐싱을 사용하여 예시 시퀀스를 생성하는 방법을 보여줍니다.

4.2 Eso-LM (B)

Eso-LM (A)는 순차 샘플링 중에는 KV 캐싱을 수행할 수 있지만, 깨끗한 토큰들이 서로 양방향 어텐션을 가지기 때문에 확산 샘플링 중에는 불가능합니다. 따라서 한 세트의 마스크 토큰이 노이즈 제거된 후, 이제 새로 노이즈 제거된 토큰들을 포함한 모든 깨끗한 토큰들에 대한 KV를 업데이트해야 합니다. 확산 샘플링 중 KV 캐싱을 가능하게 하기 위해, 우리는 깨끗한 토큰들 간에 인과적 어텐션을 강제하며, 이는 Eso-LM (B)로 이어집니다. 우리는 또한 두 단계 간에 공유 KV 캐시를 사용할 수 있도록 합니다.

4.2.1 훈련

확산 단계 노이즈 제거 트랜스포머는 노이즈를 제거할 마스크 토큰을 포함하는 $\mathbf{z}_t \sim q_t(\cdot|\mathbf{x})$을(를) 입력으로 받고, $\mathbf{x}$을(를) 목표로 받습니다. 우리는 MDM과 AO-ARM

$$A_{i,j} = \begin{cases} 0 & \text{if } \sigma^{-1}(i) \geq \sigma^{-1}(j) \quad \forall (i,j) \in [L] \times [L] \\ -\infty & \text{otherwise.} \end{cases} \tag{24}$$

예시는 그림 2를 참조하십시오. 더 나아가, A은(는) A의 행과 열을 σ로 정렬하면 인과적 어텐션 편향이 됩니다(그림 7). 우리는 이 속성을 구현에 활용합니다(부록 B.5).

순차 단계 노이즈 제거 트랜스포머는 연결된 시퀀스 $\mathbf{z}_0 \oplus \mathbf{x} \in \mathcal{V}^{2L}$을(를) 입력으로 받는데, 여기서 $\mathbf{z}_0 \sim q_0(\cdot|\mathbf{x})$은(는) 노이즈를 제거할 마스크 토큰을 포함하고, $\mathbf{x}$은(는) 목표입니다. 샘플링 중에 인과적 어텐션이 적용되고 KV 캐시가 구축되는 $\mathbf{z}_{<i} \sim p_{<i}(\phi$MDM의 깨끗한 컨텍스트는 확산 단계에서 제공됩니다. 확산 단계에서 구축된 KV 캐시를 효과적으로 사용하려면, 순차 단계 훈련 중 $\mathbf{x}^i | i \in \mathcal{C}(\mathbf{z}_0)$의 깨끗한 토큰들 간에 유사한 인과적 어텐션을 강제해야 합니다. 이를 위해, 먼저 (i) σ에서 $\mathbf{z}_0$의 깨끗한 토큰이 $\mathbf{z}_0$의 마스크 토큰보다 선행하고, (ii) σ에서 마스크 토큰이 자연스러운 순서로 배열된다는 제약 조건으로 $\sigma \sim \mathcal{P}_L$를 샘플링합니다. 그런 다음, (11)-(17)을 재사용하되, $\mathbf{x}^i | i \in \mathcal{C}(\mathbf{z}_0)$(14)에 대한 양방향 어텐션을 σ(29)에 따른 인과적 어텐션으로 대체합니다.

$$A_{i,j} = 0 \quad \text{if } i = j \quad \forall (i,j) \in \mathcal{M}(\mathbf{z}_0) \times \mathcal{M}(\mathbf{z}_0) \tag{26}$$

$$A_{i,j+L} = 0 \quad \forall (i,j) \in \mathcal{M}(\mathbf{z}_0) \times \mathcal{C}(\mathbf{z}_0) \tag{27}$$

$$A_{i,j+L} = 0 \quad \text{if } i > j \quad \forall (i,j) \in \mathcal{M}(\mathbf{z}_0) \times \mathcal{M}(\mathbf{z}_0) \tag{28}$$

$$A_{i+L,j+L} = 0 \quad \text{if } \sigma^{-1}(i) \geq \sigma^{-1}(j) \quad \forall (i,j) \in \mathcal{C}(\mathbf{z}_0) \times \mathcal{C}(\mathbf{z}_0) \tag{29}$$

$$A_{i+L,j+L} = 0 \quad \forall (i,j) \in \mathcal{M}(\mathbf{z}_0) \times \mathcal{C}(\mathbf{z}_0) \tag{30}$$

$$A_{i+L,j+L} = 0 \quad \text{if } i \geq j \quad \forall (i,j) \in \mathcal{M}(\mathbf{z}_0) \times \mathcal{M}(\mathbf{z}_0) \tag{31}$$

$$A_{i,j} = -\infty \quad \text{otherwise.} \tag{32}$$

예시는 그림 6을 참조하십시오. 4개의 L×L 블록 각각의 행과 열이 σ로 정렬될 때, A는 우리가 구현에 활용하는 구조(그림 8)를 보여줍니다(부록 B.5).

4.2.2 샘플링

4.1.2절과 동일한 설정 및 표기법에 따라, 우리는 (18)-(23)을 재사용하되, DkMDM(18)에 대한 양방향 어텐션을 σ(33)에 따른 인과적 어텐션으로 대체합니다.

Ai,j={0amp;if σ−1(i)≥σ−1(j) ∀(i,j)∈DkMDM×DkMDM₩ 0amp;∀(i,j)∈DkAR×DkMDM₩ 0amp;if i≥j ∀(i,j)∈DkAR×DkA 0amp;∀(i,j)∈Sk×(DkMDM∪DkAR)₩ 0amp;if σ−1(i)≥σ−1(j) ∀(i,j)∈Sk×Sk₩ −∞amp;otherwise.₩tag38

두 단계에서의 KV 캐싱 여기에서는 4.1.2절에서와 마찬가지로 미래 마스크 토큰 **z**ki|i∈S>k에 대한 계산을 수행하지 않습니다. 또한, 확산 또는 순차적으로 디코딩되었는지 여부에 관계없이 이전에 디코딩된 토큰들 사이에 인과적 어텐션이 있으므로, k 단계에서 새로 생성된 토큰들을 사용하여 k+1 단계에서 KV 캐시를 구축하고, k+2 단계부터 캐시된 KV 값을 사용할 수 있습니다. 그림 1은 이 샘플링 절차가 예시 시퀀스를 생성하는 방법을 보여줍니다.

---

5 실험

우리는 One Billion Words 데이터셋(LM1B)

5.1 우도 평가

저희 실험 결과에 따르면 Eso-LM은 LM1B 및 OWT에서 확산 모델 중 최신 기술 수준의 퍼플렉시티를 달성하며, MDM과 AR 퍼플렉시티 간에 더 세분화된 보간을 제공합니다 (표 1 및 표 2).

실험 설정 저희는 주요 마스크드 확산 모델(MDLM

결과 저희 방법의 경우 PPL은 (7)을 사용하여 계산됩니다. LM1B에서 저희는 α0∈0.0625,0.125,0.25,0.5,1.0으로 Eso-LM (A)를 훈련하고, α0=1으로 Eso-LM (B)를 훈련합니다. Eso-LM (A)가 MDLM과 AR 퍼플렉시티 사이를 효과적으로 보간한다는 것을 발견했습니다. 놀랍게도, 마스크 토큰에 대한 양방향 어텐션을 인과 마스크로 대체하여 MDM을 수정한 Eso-LM (A)는 α0=1에서 MDLM보다 ≈1 PPL만큼 뛰어난 성능을 보입니다. 이러한 개선은 앞서 설명한 낮은 분산 훈련 손실에 기인할 수 있습니다. 대조적으로, Eso-LM (B)는 클린 토큰 사이에 인과적 의존성을 도입하여, α0=1에서 Eso-LM (A)에 비해 성능을 ≈4 PPL만큼 저하시킵니다. 이는 Eso-LM (A)의 클린 토큰에 대한 양방향 어텐션이 Eso-LM (B)의 인과적 어텐션보다 엄격하게 더 표현력이 강하기 때문에 예상되는 결과입니다. OWT에서는 Eso-LM (B)가 확산 단계에서 KV 캐싱을 지원하는 유일한 모델(BD3-LM 포함)이라는 점에서 특히 주목할 만합니다. α0∈0.0625,0.125,0.25,0.5,1.0으로 훈련된 이 모델은 AR과 MDM 퍼플렉시티 사이의 부드러운 보간을 보여줍니다 (표 2). LM1B와 마찬가지로, OWT에서도 Eso-LM (A)는 MDLM과 경쟁력을 유지합니다.

5.2 생성 속도

저희의 정성적 평가 실험에 따르면 Eso-LM은 KV 캐싱을 지원하는 이전 확산 기반 방법보다 3−4×배 빠르며, KV 캐싱을 지원하지 않는 MDM보다 최대 65×배 빠릅니다.

실험 설정 MDLM과 같은 확산 모델은 양방향 어텐션을 사용하기 때문에 KV 캐싱을 지원하지 않습니다. BD3-LM과 같은 준자기회귀 모델은 토큰 블록을 자기회귀적으로 생성하고 완료된 블록을 캐싱함으로써 부분적인 KV 캐싱을 가능하게 하지만, 확산 중인 현재 블록에 대한 캐싱은 여전히 부족합니다. 이와 대조적으로, 저희 방법은 확산 단계에서 KV 캐싱을 고유하게 지원합니다. 저희는 저희 방법인 Eso-LM (B)의 추론 시간을 MDLM 및 BD3-LM과 컨텍스트 길이 L∈2048,8192에서 비교하며, first-hitting 샘플러

표 1: LM1B의 테스트 퍼플렉시티 (PPL; ↓). *He 등

	PPL (↓)
Autoregressive	
Transformer‡	22.83
Diffusion	
D3PM Uniform [2]	137.90$
D3PM Absorb [2]	76.90$
Diffusion-LM* [12]	118.62$
DiffusionBert [7]	63.78
SEDD Absorb‡ [15]	32.71$
SEDD Uniform$ [15]	40.25$
MDLM‡ [26]	31.78
UDLM‡ [29]	36.71
DUO‡ [27]	33.68
Interpolating diffusion and autoregressive	
BD3-LMs† [1]	
L' = 16	30.60
L' = 8	29.83
L' = 4	28.23
Eso-LM (B) (Ours)	
α0 = 1.0	35.00
Eso-LM (A) (Ours)	
α0 = 1.0	30.96
α0 = 0.5	30.51
α0 = 0.25	28.44
α0 = 0.125	25.97
α0 = 0.0625	24.51

표 2: 25만 회 훈련된 모델에 대한 OWT의 테스트 퍼플렉시티 (PPL; ↓). ‡

	PPL (↓)
Autoregressive	
Transformer	17.90$
Diffusion	
SEDD Absorb [15]	26.81$

	PPL (↓)
MDLM [26]	25.76$
UDLM [29]	30.52$
DUO [27]	27.14$
Interpolating diffusion and autoregressive	
BD3-LMs [1]	
L' = 16	23.57†
Eso-LM (A) (Ours)	
α0 = 1	26.21
Eso-LM (B) (Ours)	
α0 = 1	30.14
α0 = 0.5	27.85
α0 = 0.25	24.73
α0 = 0.125	21.87

표 3: 모든 방법에서 NFE를 L로 설정했을 때, 시퀀스 길이 L∈2048,8192에 대한 샘플링 시간(↓, 초). 보고된 값은 5회 이상의 실행에 대한 평균입니다.

Method	L = 2048	L = 8192
AR (Oracle)	13.30.9	54.00.2
MDLM	201.30.4	5438.33.3
BD3-LM (L' = 4)	24.30.7	312.01.7
BD3-LM (L' = 16)	21.30.1	268.11.2
Eso-LM (B)	14.60.3	82.10.3

결과 표 3에서 볼 수 있듯이, KV 캐싱이 없는 MDLM과 비교할 때 Eso-LM (B)는

L=2048에서 ~14×배, L=8192에서 ~65×배 더 빠릅니다. 부분적으로 캐싱을 지원하는 BD3-LM과 비교했을 때, Eso-LM (B)는 L=8192에서 BD3-LM (L′=16)보다 ~3.2×배 빠르고 BD3-LM (L′=4)보다 ~3.8×배 빠릅니다. 이러한 속도 향상은 KV 캐싱과 함께, 디노이징될 마스크된 토큰과 이전에 예측된 토큰으로 순방향 패스를 제한하여 중복 계산을 피하는 스케줄러 S에서 비롯됩니다—이는 BD3-LM에는 없는 기능입니다. NFE를 L으로 제한함에 따라, 저희 방법은 AR 모델보다 느린데, 이는 KV 재사용이 마지막에서 두 번째 디노이징 단계에서만 가능하기 때문입니다. 그럼에도 불구하고 전반적인 효율성은 이전 확산 기반 기준선을 능가합니다 (섹션 4.2.2 참조).

5.3 생성 품질

저희의 샘플링 실험 결과에 따르면 (1) Eso-LM은 샘플링 속도와 품질의 파레토 프론티어에서 새로운 SOTA를 확립하고 (그림 4), (2) 높은 NFE에서 모든 이전 확산 모델보다 더 높은 품질의 샘플을 생성하며, (3) 이전 보간 확산 방법인 BD3-LM과는 달리 낮은 NFE에서 모드 붕괴를 겪지 않습니다 (그림 3).

실험 설정 저희는 α0train∈0.125,0.25,1으로 Eso-LM (B)를 훈련하고, NFE를 제어하기 위해 다양한 (α0eval,T)∈0.0625,0.25,0.5,1×16,128,1024로 샘플을 생성합니다 (NFE = |S|). MDLM 및 BD3-LM은 Sahoo 등

---

img-4.jpeg Figure 3: Gen. PPL vs. NFEs for models trained on OWT. Entropy values are in Tables (8, 9, 10).

img-5.jpeg Figure 4: Eso-LM (B) establishes SOTA on the Pareto frontier of sampling speed and quality.

1 미만이기 때문입니다. 저희는 샘플 품질을 위해 생성 퍼플렉시티(GPT-2 Large를 통해)를 측정하고, 다양성을 위해 평균 엔트로피

생성 PPL 대 NFE의 파레토 프론티어 그림 3에서 볼 수 있듯이, 높은 NFE(≈ 1024)에서 Eso-LM (B)는 MDLM 및 BD3-LM보다 성능이 뛰어나며 AR 모델 품질에 거의 근접합니다. NFE가 감소함에 따라 BD3-LM은 teacher-forced 훈련 때문에 성능이 크게 저하됩니다. 즉, 훈련 중에는 미래 블록이 깨끗한 이전 블록에 의해 조건화되지만, 평가 중에는 이전 블록 예측의 오류가 샘플 품질의 상당한 저하로 이어집니다. Eso-LM은 이 문제를 피합니다. 예상대로 낮은 NFE 영역에서는 더 높은 α0train (훈련 중 "더 많은 확산")이 품질을 향상시킵니다: α0train=0.125,0.25,1 중에서는 α0train=1가 가장 좋은 성능을 보이며, MDLM과 경쟁력을 유지합니다. 전반적으로, Eso-LM (B)는 낮은 NFE에서는 MDLM과, 높은 NFE에서는 AR 모델과 경쟁력을 갖습니다—BD3-LM과는 다릅니다.

생성 PPL 대 속도의 파레토 프론티어 샘플링 효율성을 비교하기 위해, 각 방법이 단일 샘플을 생성하는 데 걸리는 중앙값 샘플링 시간(5회 시도에 걸쳐)을 초 단위로 기록하고, 그림 4에 생성 PPL 대 샘플링 시간을 플롯합니다. 무한 시도의 한계에서 중앙값 샘플링 시간은 NFE의 증가 함수이며, 사용된 방법 및 샘플링 하이퍼파라미터에 의해 조절됩니다. 각 방법에 대해, 저희는 모든 구성에 걸쳐 파레토 프론티어를 구성합니다: Eso-LM (B) (α0train, α0eval 및 T에 대해), BD3-LM (L′ 및 T에 대해), MDLM (T에 대해). 저희는 Eso-LM (B)가 속도-품질 파레토 프론티어에서 새로운 최신 기술 수준을 확립한다는 것을 발견했습니다. 샘플링 시간에 대한 예산이 적을 경우, α0train=1으로 훈련된 모델이 가장 좋고, 예산이 많을 경우, α0train=0.125 모델은 AR에 가까운 품질을 생성합니다. 중간 값(예: α0train=0.25,0.5)은 속도와 품질 사이에서 최적의 균형을 제공합니다.

6 관련 연구, 논의 및 결론

AR 모델 AR 모델은 토큰을 왼쪽에서 오른쪽으로 생성하며 품질 면에서는 최신 기술 수준을 유지하지만, 느리고 순차적인 추론과 제한된 제어 가능성으로 인해 어려움을 겪습니다. 이와 대조적으로, Eso-LM은 순차적 단계에서 AR과 유사한 생성과 초기 확산 단계에서 임의 순서의 병렬 생성을 결합합니다. 확산 중, Eso-LM (B)는 이전에는 AR 모델에만 국한되었던 KV 캐싱

마스크드 확산 MDM [26, 30]은 단계당 여러 토큰을 생성하지만, 비용이 많이 드는 양방향 어텐션에 의존합니다. Eso-LM은 두 가지 방식으로 효율성을 개선합니다: Eso-LM (A)는 어텐션을 깨끗하고 디노이징이 예정된 토큰으로 제한하며, Eso-LM (B)는 KV 캐싱을 위해 AO-ARM

블록 확산 BD3-LM

훈련 중 teacher forcing 사용으로 인해 후속 블록의 샘플 품질에 심각한 영향을 미칩니다. Eso-LM은 이 문제로 고통받지 않습니다.

결론

저희는 자기회귀(AR) 모델과 마스크드 확산 모델(MDM)을 완벽하게 융합하여 샘플 품질과 생성 속도 면에서 두 모델 사이의 효과적인 보간을 가능하게 하는 언어 모델링의 새로운 패러다임을 제시합니다. 저희 방법은 병렬 생성을 유지하면서 MDM에서 KV 캐싱을 가능하게 한 최초의 방법으로, 추론 속도를 크게 가속화합니다. 이는 속도와 정확도 면에서 블록 확산 방법을 능가하며, 언어 모델링 벤치마크에서 새로운 최신 기술 수준을 확립합니다. 언어 모델링 연구를 수행하고 있는 만큼, 이 연구 분야에 내재된 위험과 기회를 함께 가지고 있습니다.

참고 문헌

- [1] Marianne Arriola, Subham Sekhar Sahoo, Aaron Gokaslan, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Justin T Chiu, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. In The Thirteenth International Conference on Learning Representations, 2025. URL https://openreview.net/forum?id=tyEyYT267x.
- [2] Jacob Austin, Daniel D Johnson, Jonathan Ho, Daniel Tarlow, and Rianne Van Den Berg. Structured denoising diffusion models in discrete state-spaces. Advances in Neural Information Processing Systems, 34:17981-17993, 2021.
- [3] Ciprian Chelba, Tomas Mikolov, Mike Schuster, Qi Ge, Thorsten Brants, Phillipp Koehn, and Tony Robinson. One billion word benchmark for measuring progress in statistical language modeling, 2014.
- [4] Arman Cohan, Franck Dernoncourt, Doo Soon Kim, Trung Bui, Seokhwan Kim, Walter Chang, and Nazli Goharian. A discourse-aware attention model for abstractive summarization of long documents. Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers), 2018. doi: 10.18653/v1/n18-2097. URL http://dx.doi.org/10.18653/v1/n18-2097.
- [5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805, 2018.
- [6] Aaron Gokaslan, Vanya Cohen, Ellie Pavlick, and Stefanie Tellex. Openwebtext corpus. http://Skylion007.github.io/OpenWebTextCorpus, 2019.
- [7] Zhengfu He, Tianxiang Sun, Kuanning Wang, Xuanjing Huang, and Xipeng Qiu. Diffusion-bert: Improving generative masked language models with diffusion models. arXiv preprint arXiv:2211.15029, 2022.
- [8] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. Advances in neural information processing systems, 33:6840-6851, 2020.
- [9] Diederik Kingma, Tim Salimans, Ben Poole, and Jonathan Ho. Variational diffusion models. Advances in neural information processing systems, 34:21696-21707, 2021.
- [10] Diederik P Kingma and Max Welling. Auto-encoding variational {Bayes}. In ICLR, 2014.
- [11] Seul Lee, Karsten Kreis, Srimukh Prasad Veccham, Meng Liu, Danny Reidenbach, Yuxing Peng, Saee Paliwal, Weili Nie, and Arash Vahdat. Genmol: A drug discovery generalist with discrete diffusion. arXiv preprint arXiv:2501.06158, 2025.
- [12] Xiang Li, John Thickstun, Ishaan Gulrajani, Percy S Liang, and Tatsunori B Hashimoto. Diffusion-lm improves controllable text generation. Advances in Neural Information Processing Systems, 35:4328-4343, 2022.
- [13] Anji Liu, Oliver Broadrick, Mathias Niepert, and Guy Van den Broeck. Discrete copula diffusion.

[14] Chengyi Liu, Wenqi Fan, Yunqing Liu, Jiatong Li, Hang Li, Hui Liu, Jiliang Tang, and Qing Li. Generative diffusion models on graphs: Methods and applications. arXiv preprint arXiv:2302.02591, 2023.

- [15] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. arXiv preprint arXiv:2310.16834, 2024.
- [16] Mitch Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. Building a large annotated corpus of english: The penn treebank. Computational linguistics, 19(2):313-330, 1993.
- [17] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016.
- [18] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. arXiv preprint arXiv:2502.09992, 2025.
- [19] Hunter Nisonoff, Junhao Xiong, Stephan Allenspach, and Jennifer Listgarten. Unlocking guidance for discrete state-space diffusion and flow models. arXiv preprint arXiv:2406.01572, 2024.
- [20] Jingyang Ou, Shen Nie, Kaiwen Xue, Fengqi Zhu, Jiacheng Sun, Zhenguo Li, and Chongxuan Li. Your absorbing discrete diffusion secretly models the conditional distributions of clean data. In The Thirteenth International Conference on Learning Representations, 2025. URL https://openreview.net/forum?id=sMyXP8Tanm.
- [21] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernandez. The LAMBADA dataset: Word prediction requiring a broad discourse context. In Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 1525-1534, Berlin, Germany, August 2016. Association for Computational Linguistics. URL http://www.aclweb.org/anthology/P16-1144.
- [22] William Peebles and Saining Xie. Scalable diffusion models with transformers. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 4195-4205, 2023.
- [23] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference, 2022. URL https://arxiv.org/abs/2211.05102.
- [24] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.
- [25] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. Journal of machine learning research, 21(140):1-67, 2020.
- [26] Subham Sekhar Sahoo, Marianne Arriola, Aaron Gokaslan, Edgar Mariano Marroquin, Alexander M Rush, Yair Schiff, Justin T Chiu, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. In The Thirty-eighth Annual Conference on Neural Information Processing Systems, 2024. URL https://openreview.net/forum?id=L4uaAR4ArM.
- [27] Subham Sekhar Sahoo, Justin Deschenaux, Aaron Gokaslan, Guanghan Wang, Justin T Chiu, and Volodymyr Kuleshov. The diffusion duality. In ICLR 2025 Workshop on Deep Generative Model in Machine Learning: Theory, Principle and Efficacy, 2025. URL https://openreview.net/forum?id=CB0Ub2yXjC.
- [28] Yair Schiff, Chia-Hsiang Kao, Aaron Gokaslan, Tri Dao, Albert Gu, and Volodymyr Kuleshov. Caduceus: Bi-directional equivariant long-range dna sequence modeling. arXiv preprint arXiv:2403.03234, 2024.

[29] Yair Schiff, Subham Sekhar Sahoo, Hao Phung, Guanghan Wang, Sam Boshar, Hugo Dallatorre, Bernardo P de Almeida, Alexander M Rush, Thomas PIERROT, and Volodymyr Kuleshov. Simple guidance mechanisms for discrete diffusion models. In The Thirteenth International Conference on Learning Representations, 2025. URL https://openreview.net/forum?id=i5MrJ6g5G1.

- [30] Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis K. Titsias. Simplified and generalized masked diffusion for discrete data, 2025. URL https://arxiv.org/abs/2406.04329.
- [31] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In International conference on machine learning, pp. 2256-2265. PMLR, 2015.
- [32] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. Advances in neural information processing systems, 32, 2019.
- [33] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. arXiv preprint arXiv:2104.09864, 2021.
- [34] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. Advances in neural information processing systems, 30, 2017.
- [35] Guanghan Wang, Yair Schiff, Subham Sahoo, and Volodymyr Kuleshov. Remasking discrete diffusion models with inference-time scaling. arXiv preprint arXiv:2503.00307, 2025.
- [36] Xiang Zhang, Junbo Jake Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In NIPS, 2015.
- [37] Kaiwen Zheng, Yongxin Chen, Hanzi Mao, Ming-Yu Liu, Jun Zhu, and Qinsheng Zhang. Masked diffusion models are secretly time-agnostic masked models and exploit inaccurate categorical sampling. arXiv preprint arXiv:2409.02908, 2024.

목차

1 서론 1 2 배경 2 2.1 자기회귀 모델 3 2.2 마스크 확산 모델 3 2.3 블록 노이즈 제거 확산 이산 언어 모델 3 3 난해한 언어 모델 4 3.1 자기회귀 모델과 마스크 확산 융합 4 3.2 샘플링 5 4 공유 노이즈 제거 트랜스포머를 위한 어텐션 메커니즘 설계 5 4.1 Eso-LM (A) 6 4.2 Eso-LM (B) 8 5 실험 9 5.1 우도 평가 9 5.2 생성 속도 9 5.3 생성 품질 10 6 관련 연구, 논의 및 결론 11 부록 16 부록 A 배경 16 A.1 BD3-LM 하이퍼파라미터 T′ 및 num_tries 16 부록 B 난해한 언어 모델 17 B.1 MDM 손실 17 B.2 학습 절차 18 B.3 순차 단계의 어텐션 편향 18 B.4 샘플링 일정 및 샘플러 18 B.5 구현 세부 사항 19 B.6 실험 세부 사항 20 부록 C 추가 실험 22 C.1 분할 비율에 대한 절제 연구 22 C.2 OWT 혼란도 22 C.3 제로샷 우도 평가 22 C.4 OWT로 학습된 모델에 의해 생성된 샘플의 생성 혼란도 및 엔트로피 23

부록

부록 A 배경

A.1 BD3-LM 하이퍼파라미터 T′ 및 num_tries

BD3-LM

또한, BD3-LM의 코드베이스가 샘플을 선별한다는 것을 발견했다. 더 구체적으로, 단일 샘플을 생성하기 위해 코드베이스는 하나의 샘플이 특정 품질 관리 테스트를 통과할 때까지 새로운 샘플을 계속 생성한다(num_tries 횟수만큼). 기본적으로 num_tries는 10이며, 10번의 시도가 모두 소진되고 통과하는 샘플이 없을 경우 코드베이스는 샘플링 실패를 보고한다. 경험적으로, T′=5000에 대해서는 샘플링 실패가 발생하지 않는다는 것을 발견했다.

병렬 생성에 대한 BD3-LM의 실제 성능을 조사하기 위해 num_tries = 1로 설정하고 품질 관리 테스트를 비활성화하며 광범위한 T 값에 걸쳐 BD3-LM의 샘플을 평가했다(그림 5). 여기서 그리고 그림 5에서, T은 BD3-LM의 모든 블록에 대한 샘플링 단계의 합계를 의미한다. 예를 들어, L′=16와 T=4096은 블록당 T′=4096/(1024/16)=64 샘플링 단계가 사용됨을 의미한다. 대조적으로, BD3-LM의 코드베이스는 기본적으로 T′=5000를 사용하며, 이는 그림 5의 T=∞에 해당한다. MDLM의 경우, T는 블록이 없으므로 정상적으로 해석될 수 있다.

그림 5에서 보듯이, 더 많은 병렬 생성을 가능하게 하기 위해 T이 감소함에 따라 BD3-LM의 샘플 품질과 샘플 다양성 모두 MDLM보다 현저히 나빠진다. 이러한 성능 저하는 섹션 6에서 논의된 모드 붕괴 문제 때문이다. 또한 num_tries를 늘리면 BD3-LM의 샘플 엔트로피가 다소 향상되고(표 4의 두 번째 행) 모드 붕괴가 완화될 수 있지만, AR 및 MDLM에는 거의 또는 전혀 개선이 되지 않는다는 것을 발견했다.

img-6.jpeg Figure 5: Gen. Perplexity (↓) with nucleus sampling (p=0.9) against the number of sampling steps for AR, MDLM and BD3-LMs trained for 1 million steps. The number of sampling steps for AR is always 1024; we extend it to other values for easier comparison. The number next to each data point records its sample entropy (↑) ; a value less than 5 usually indicates mode collapse.

표 4: BD3-LM L′=16, MDLM 및 핵 샘플링 p=0.9을 사용하는 AR의 생성 PPL(↓) 및 엔트로피(↑) (괄호 안). BD3-LM

	BD3LM		**MDLM**		**AR**	
num_tries	1	10	1	10	1	10
T=1024	72.80 (5.35)	77.71 (5.41)	41.92 (5.36)	41.79 (5.37)	13.03 (5.26)	13.76 (5.32)
T=256	356.02 (5.11)	440.69 (5.28)	45.07 (5.40)	44.57 (5.39)	13.03 (5.26)	13.76 (5.32)

부록 B 난해한 언어 모델

B.1 MDM 손실

NLL은 다음과 같이 주어진다:

−logpθ(x)≤−Ez0~q0(·|x)logpθAR(x|z0)+DKL(q0(z0|x)|pθMDM(z0)) =−Ez0~q0(·|x) [Σℓ∈M(z0)logpθAR(xℓ|z0,x<ℓ)]+DKL(q0(z0|x)|pθMDM(z0)).₩tag39

$\mathbf{z}0$는 인덱스 ℓ를 초과하는 인덱스에 깨끗한 토큰을 포함할 수 있음에 유의하라. 섹션 3.1에서 논의된 바와 같이, AR 로그 우도는 다음과 같이 주어진다:

Σℓ∈M(z0)logpθAR(xℓ|z0,x<ℓ)=Σℓ∈M(z0)log(xθℓ(z0⊙x<ℓ),xℓ),₩tag40

여기서 우리는 마스크된 인덱스 $\mathscr{M}(\mathbf{z}0)$에서만 손실을 계산한다.

(39)의 KL 항을 계산하기 위해, 우리는 $\mathbf{z}$0에 대한 마스크 확산 프로세스를 정의한다. 이 확산 프로세스에서 시간 t∈[0,1]에서의 전방 주변 분포 $\mathbf{z}$tℓ~qt(·|$\mathbf{x}$ℓ)는 (1)과 동일하지만, 축소된 범위 (αt)t∈[0,1]∈[0,α0] (αt=0=α0를 가지며 $\mathbf{z}$t=0=$\mathbf{z}$0이고 αt=1=0를 가지며 $\mathbf{z}$t=1=$\mathbf{m}$1:L인 t에 대한 엄격하게 감소하는 함수)를 가진 노이즈 스케줄을 사용한다. T 확산 단계를 통해, 우리는 다음을 얻는다:

DKL(q0(z0|x)|pθMDM(z0))=Ez0[logq0(z0|x)pθMDM(z0)] =Ez0[logEz1T:1[q(z0:1|x)pθMDM(z0:1)]] ≤Ez0:1[logq(z0:1|x)pθMDM(z0:1)] =Ez0:1[Σt∈{1T,2T,⋯,1}logq(zt−1T|zt,x)pθMDM(zt−1T|zt)] =ΣtEzt[DKL(q(zt−1T|zt,x)|pθMDM(zt−1T|zt))]

Sahoo 등

=Et~U[0,1],zt~qt[αt′1−αtΣℓ∈M(zt)log(xθℓ(zt),xℓ)].₩tag41

마지막으로, (40)과 (41)을 결합하여, 우리는 원하는 결과를 얻는다:

LNELBO(x;θ) =Ez0~q0[−Σℓ∈M(z0)log(xθℓ(z0⊙x<ℓ),xℓ)⏟A R l o s s] +∫t=0t=1αt′1−αtEzt~qt[Σℓ∈M(zt)log(xθℓ(zt),xℓ)]dt⏟M D M l o s s.₩tag42

B.2 학습 절차

알고리즘 1은 전체 학습 절차를 설명한다.

알고리즘 1 Eso-LMs 훈련 입력: 데이터셋 D, 배치 크기 bs, 순방향 노이즈 프로세스 qt(·|$\mathbf{x}$), 모델 $\mathbf{x}$θ, 학습률 η 수렴하지 않을 때까지 반복 $\mathbf{x}$1,$\mathbf{x}$<i>2,⋯,$\mathbf{x}$bs~D i←1부터 bs/2까지 반복 ▷ 만약 α0=1이면, 1부터 bs까지 반복. $\mathbf{z}$0~q0(·|$\mathbf{x}$) σ~$\mathscr{P}$L 제약 조건 포함. A에서 어텐션 편향 $\mathbf{x}$θ 구성에 사용됨 (섹션 4) $\mathscr{L}$i←−Σℓ∈$\mathscr{M}$($\mathbf{z}$0)log($\mathbf{x}$θℓ($\mathbf{z}$0,$\mathbf{x}$<ℓ),$\mathbf{x}$iℓ) (7)에 따른 순차적 손실 추정량 반복 종료 i←bs/2+1부터 bs까지 반복 ▷ 만약 α0=1이면, 이 반복을 건너뛴다. 샘플 t~$\mathscr{U}$[0,1] $\mathbf{z}$t~qt(·|$\mathbf{x}$) σ~$\mathscr{P}$L 제약 조건 포함. A에서 어텐션 편향 $\mathbf{x}$θ 구성에 사용됨 (섹션 4) $\mathscr{L}$i←αi′1−αiΣℓ∈$\mathscr{M}$($\mathbf{z}$t)log($\mathbf{x}$θℓ($\mathbf{z}$t),$\mathbf{x}$iℓ) (7)에 따른 MDM 손실 추정량 반복 종료 θ←θ−η∇θΣi=1bs$\mathscr{A}$ 반복 종료

B.3 순차 단계의 어텐션 편향

그림 6은 순차 단계 학습을 위한 어텐션 편향 비교를 보여줍니다.

img-7.jpeg Figure 6: Comparison of attention biases for sequential-phase training. Orange represents 0 (attention) and gray represents $-\infty$ (no attention). The clean sequence is $\mathbf{x}$=(A,B,C,D,E,F) and hence L=6 . After random masking, we obtain $\mathbf{z}0$=(A,M,C,M,M,F) . The integers denote the position indices with $\mathscr{M}(\mathbf{z}0)$=2,4,5 and $\mathscr{C}(\mathbf{z}0)$=1,3,6 . The random ordering among $\mathscr{C}(\mathbf{z}0)$ is (3,1,6) . Red highlights the differences between Eso-LM (A) and Eso-LM (B).

B.4 샘플링 스케줄 및 샘플러

Eso-LM은 확산 단계와 순차 단계의 두 가지 샘플링 단계를 수행합니다. 확산 단계 내에서는 토큰이 무작위 순서로, 잠재적으로는 병렬로 노이즈 제거됩니다. 순차 단계 내에서는 남아있는 마스크 토큰이 왼쪽에서 오른쪽으로, 한 번에 하나씩 순차적으로 노이즈 제거됩니다.

먼저, (i) 확산 단계 동안 노이즈 제거할 총 토큰 수와 (ii) 확산 단계당 노이즈 제거할 토큰 수를 결정하기 위해, 우리는

T 단계. 각 단계마다, 시간 t에 노이즈를 제거할 토큰 수를 다음과 같이 계산합니다.

nt=Binom(n=ntremaining,p=αs−αt1−αt), (43)

여기서 s=t−dt 및 ntremaining=L−Σt′>tnt′입니다. T가 클 때, 일부 nt들은 0이 될 수 있습니다. 이 알고리즘에 의해 생성된 모든 nt들은 정렬된 리스트에 수집되며, 0인 nt들은 제외됩니다. 우리는 모든 nt들의 합을 nMDM로 나타내고 nAR=L−nMDM로 정의합니다.

우리는 확산을 통해 노이즈 제거할 [L]에서 nMDM개의 토큰 인덱스를 선택하고, 토큰 인덱스의 보완적인 부분 집합을 순차적으로 노이즈 제거하는 데 사용합니다. 예를 들어, L=8이고 토큰 인덱스가 [1,2,⋯,8]이라고 가정합니다. 위 알고리즘에서 nMDM=5를 얻었다고 가정합니다. 그러면 우리가 선택할 수 있는 확산 인덱스는 (1,3,4,6,7)이고 보완적인 순차 인덱스는 (2,5,8)입니다. 우리는 또한 확산 인덱스를 무작위로 재배열하여 예를 들어 (3,1,6,4,7)로 만들어 무작위 순서 노이즈 제거를 수행합니다.

0이 아닌 nt들의 목록과 순열된 확산 인덱스 집합이 주어지면, nt들을 기준으로 확산 인덱스를 분할하여 확산 샘플링 스케줄을 생성합니다. 0이 아닌 nt들의 목록이 (2,1,2)이라고 가정합시다. 이를 사용하여 순열된 확산 인덱스 집합 (3,1,6,4,7)를 분할하면 확산 단계에 대한 다음 샘플링 스케줄 $\mathcal{S}$MDM=((3,1),(6),(4,7))를 얻습니다. 순차 단계에 대한 노이즈 제거 스케줄은 단순히 $\mathcal{S}$AR=((2),(5),(8))입니다. 통합 샘플링 스케줄 $\mathcal{S}$은 $\mathcal{S}$MDM와 $\mathcal{S}$AR의 연결입니다. 이 예시에서, $\mathcal{S}$=(S1,S2,S3,S4,S5,S6)는 S1=(3,1),S2=(6),S3=(4,7),S4=(2),S5=(5) 및 S6=(8)입니다. 이는 6개의 NFE에 해당합니다. 마지막으로, $\mathcal{S}$는 Algo. 2에 전달되며, 나머지 샘플링 절차를 처리합니다. 섹션 4.1.2 및 섹션 4.2.2에서 논의된 노이즈 제거 순서 σ와 연결하면, 이 예시에서 σ=(3,1,6,4,7,2,5,8)를 가집니다.

입력: 시퀀스 길이 L, 통합 샘플링 스케줄 $\mathcal{S}$ $\mathbf{z}$=[MASK_INDEX,⋯,MASK_INDEX] C= ▷ i←1부터 |$\mathcal{S}$|까지 클린 토큰 인덱스 ▷ |C|≥nMDM 로짓일 때 순차적 작업은 자동으로 발생 ←$\mathbf{x}$θ($\mathbf{z}$[C∪Si]) ▷ 비고 참조 로짓 ← Si에 해당하는 로짓 선택 $\mathbf{z}$[Si]←categorical_sample(logits, dim=-1) ▷ 로짓은 (|Si|,|$\mathscr{V}$|) 모양을 가짐 C←C∪Si 반복 종료 반환: $\mathbf{z}$ 비고. $\mathbf{z}$[C∪Si]는 노이즈 제거 모델 $\mathbf{x}$θ에 입력되는 $\mathbf{z}$ 내 토큰의 부분 집합을 나타냅니다. 토큰 $\mathbf{z}\ell$∈$\mathbf{z}$[C∪Si]에 대한 위치 임베딩은 원래 시퀀스 $\mathbf{z}$의 그것과 동일하도록 보장됩니다. Eso-LM (A) 및 Eso-LM (B)에 대한 샘플링 어텐션

편향 A 계산에 대해서는 각각 섹션 4.1.2 및 섹션 4.2.2를 참조하십시오. Eso-LM (B)의 경우, 인과적 어텐션 사용으로 인해 $\mathbf{x}_\theta$는 클린 토큰이 처음 처리될 때 KV-값을 캐시할 수 있습니다.

알고리즘 2 Eso-LMs 샘플링

B.5 구현 세부 사항

B.5.1 훈련

확산 단계

Eso-LM (A)와 Eso-LM (B) 모두, 입력 $\mathbf{z}_t$와 A의 행과 열을 σ에 따라 정렬합니다. 이는 효율적으로 구현될 수 있는 고전적인 어텐션 편향(그림 7)을 만듭니다. 우리는 또한 $\mathbf{z}_t$의 위치 임베딩을 σ에 따라 정렬하여 토큰들이 원래의 위치 임베딩을 유지하도록 합니다. 손실 계산 시, 우리는 대상 $\mathbf{x}$도 σ에 따라 추가로 정렬합니다.

순차 단계

Eso-LM (A)와 Eso-LM (B) 모두, $\mathbf{z}_0$, $\mathbf{x}$ (입력 $\mathbf{z}_0 \oplus \mathbf{x}$로 연결하기 전), 그리고 A의 네 L×L 블록 각각의 행과 열을 σ에 따라 정렬합니다. 또한 $\mathbf{z}_0$의 클린 토큰에 대해 특정 추가 어텐션 연결을 추가합니다. 이는 다른 토큰이 $\mathbf{z}_0$의 클린 토큰에 주의를 기울이지 않으므로 트랜스포머 출력에 영향을 미치지 않습니다. 이 두 가지 변환은 희소하고 패턴화된 블록 구조(그림 8)를 가진 어텐션 편향을 만들어 효율적으로 구현할 수 있습니다. 우리는 또한 $\mathbf{z}_0$와 $\mathbf{x}$의 위치 임베딩을 σ에 따라 정렬하여 토큰들이 원래의 위치 임베딩을 유지하도록 합니다. 손실 계산 시, 우리는 대상 $\mathbf{x}$도 σ에 따라 추가로 정렬합니다.

Figure 7: Comparison of attention biases for diffusion-phase training, before and after sorting the rows and columns by σ. Orange represents 0 (attention) and gray represents $-\infty$ (no attention). The clean sequence is $\mathbf{x}$=(A,B,C,D,E,F) and hence L=6. After random masking, we obtain $\mathbf{z}_t$=(A,M,C,M,M,F). The integers denote the position indices with $\mathcal{M}(\mathbf{z}_t)$=2,4,5 and $\mathcal{C}(\mathbf{z}_t)$=1,3,6. The random ordering sampled is σ=(3,1,6,4,5,2)~$\mathcal{P}$6 with clean tokens before mask tokens. Red highlights the differences between Eso-LM (A) and Eso-LM (B).

Figure 8: Comparison of attention biases for sequential-phase training, before and after sorting the rows and columns of each of the four L×L blocks by σ. Orange represents 0 (attention) and gray represents $-\infty$ (no attention). The clean sequence is $\mathbf{x}$=(A,B,C,D,E,F) and hence L=6. After random masking, we obtain $\mathbf{z}_0$=(A,M,C,M,M,F). The integers denote the position indices with $\mathcal{M}(\mathbf{z}_0)$=2,4,5 and $\mathcal{C}(\mathbf{z}_0)$=1,3,6. The random ordering among $\mathcal{C}(\mathbf{z}_0)$ is (3,1,6). Red highlights the differences between Eso-LM (A) and Eso-LM (B). Green highlights the extra connections added for clean tokens in $\mathbf{z}_0$ so that the attention biases display useful patterns after sorting - they don't affect the transformer output because no other token attends to clean tokens in $\mathbf{z}_0$.

B.5.2 샘플링

그림 9는 Eso-LM (A)의 샘플링 절차가 예시 시퀀스를 생성하는 방법을 보여줍니다. 그림 10은 Eso-LM (B)의 샘플링 절차가 예시 시퀀스를 생성하는 방법을 보여줍니다.

B.6 실험 세부 사항

B.6.1 저분산 샘플러

훈련 중 분산을 줄이기 위해 Kingma et al.

B.6.2 우도 평가

우도를 평가하기 위해 t에 대한 단일 몬테카를로 추정치를 사용합니다. 추정치의 분산을 줄이기 위해 저분산 샘플러

---

img-10.jpeg Figure 9: Efficient generation of an example sequence with Eso-LM (A). During Diffusion Phase, Eso-LMs denoise one or more, potentially non- neighboring mask tokens (M) per step. During Sequential Phase, Eso-LMs denoise the remaining mask tokens one at a time from left to right. Eso-LM (A) doesn't process mask tokens not scheduled for denoising and allows for KV caching in sequential phase: blue bounding boxes enclose transformer cells that are building their KV cache; a cell becomes blue once its KV cache is built. The sequences below the transformers depict tokens in their natural order.

img-11.jpeg Figure 10: (Copy of Fig. 1) Efficient generation of an example sequence with Eso-LM (B). During Diffusion Phase, Eso-LMs denoise one or more, potentially non- neighboring mask tokens (M) per step. During Sequential Phase, Eso-LMs denoise the remaining mask tokens one at a time from left to right. Eso-LM (B) allows for KV caching in both phases using just a single unified KV cache: blue bounding boxes enclose transformer cells that are building their KV cache; a cell becomes blue once its KV cache is built. The sequences below the transformers depict tokens in their natural order.

B.6.3 언어 모델링

우리는 [15, 26]에 따라 One Billion Words 데이터셋을 디토큰화했으며, 해당 코드는 여기서 찾을 수 있습니다. 우리는 Austin et al.

우리는 GPT2 토크나이저로 OpenWebText를 토큰화합니다. 시퀀스를 연결하고 래핑하여 길이를 1,024로 만듭니다. 래핑 시, 연결된 시퀀스 사이에 eos 토큰을 추가합니다. One Billion Words와 달리 OpenWebText에 대한 최종 전처리된 시퀀스는 첫 번째 및 마지막 토큰으로 특수 토큰을 가지지 않습니다. OpenWebText는 검증 분할이 없으므로, 마지막 10만 개의 문서를 검증용으로 남겨둡니다.

Eso-LMs는 우리 자기회귀 기준 모델인 SEDD, MDLM, UDLM, DUO와 동일한 매개변수화를 공유합니다: Lou et al.

우리는 로그-선형 노이즈 스케줄 $\alpha t = \alpha 0(1-t)$를 사용합니다. 배치 크기 512, 0에서 3e-4까지 2,500단계 동안 학습률 웜업, AdamW 옵티마이저를 사용합니다. One Billion Words에 대해 1M 단계, OpenWebText에 대해 250K 단계 동안 상수 학습률을 사용합니다. 드롭아웃 비율은 0.1입니다. 모델은 H200 GPU에서 훈련합니다. One Billion Words에서 1M 단계 동안 훈련하는 데 ~21일 때 $\alpha 0=1$시간, ~34일 때 $\alpha 0<1$시간이 걸립니다. OpenWebText에서 250K 단계 동안 훈련하는 데 ~50일 때 $\alpha 0=1$시간, ~69일 때 $\alpha 0<1$시간이 걸립니다.

부록 C 추가 실험

C.1 분할 비율에 대한 절제 연구

표 5를 참조하십시오.

표 5: 500K 단계로 훈련된 Eso-LMs (A)의 LM1B 테스트 퍼플렉시티 (↓)와 훈련 중 (7)의 MDM 손실 평가에 사용된 각 배치 내 예시 비율 비교. 각 배치 내 나머지 예시들은 훈련 중 (7)의 AR 손실 평가에 사용됩니다.

	0.75	**0.5**	**0.25**	**0.125**
Eso-LM (A)				
$\alpha 0=0.5$	32.25	31.53	Diverged	Diverged
$\alpha 0=0.25$	30.49	29.33	Diverged	Diverged
$\alpha 0=0.125$	27.76	26.73	Diverged	Diverged
$\alpha 0=0.0625$	25.92	25.07	Diverged	Diverged

C.2 OWT 퍼플렉시티

표 6을 참조하십시오.

표 6: 250K 및 1M 단계로 훈련된 모델의 OWT 테스트 퍼플렉시티 (PPL; ↓). 확산 모델 및 보간 방법에 대한 경계를 보고합니다. ǂ

	PPL 250K (↓)	**PPL 1M (↓)**
Autoregressive		
Transformer	17.90$	17.54$
Diffusion		
SEDD Absorb [15]	26.81$	24.10$
MDLM [26]	25.76$	22.98$
UDLM [29]	30.52$	27.43
DUO [27]	27.14$	25.20
Interpolating diffusion and autoregressive		

	PPL 250K (↓)	PPL 1M (↓)
BD3-LMs [1]		
L' = 16	23.57†	22.27‡
Eso-LM (A) (Ours)		
α0 = 1	26.21	-
Eso-LM (B) (Ours)		
α0 = 1	30.14	-
α0 = 0.5	27.85	-
α0 = 0.25	24.73	-
α0 = 0.125	21.87	-

C.3 제로샷 우도 평가

우리는 또한 OWT에서 훈련된 모델을 사용하여 보이지 않는 데이터셋을 얼마나 잘 모델링하는지 평가하여 모델의 일반화 능력을 탐색합니다 (표 7). 우리 Eso-LMs의 퍼플렉시티를 SEDD

---

[2], MDLM [26], BD3-LMs [1], and an AR Transformer language model. Our zero-shot datasets include the validation splits of Penn Tree Bank (PTB; [16]), Wikitext [17], LM1B, Lambada [21], AG News [36], and Scientific Papers (Pubmed and Arxiv subsets; [4]).

표 7: OWT에서 250K 단계로 훈련된 모델의 제로샷 퍼플렉시티 (↓). 확산 모델 및 보간 방법에 대한 경계를 보고합니다. AR에 대한 수치는

	PTB	Wikitext	LM1B	Lambada	AG News	Pubmed	Arxiv
AR	81.07	25.32	51.14	52.13	52.11	48.59	41.22
MDLM	93.82	36.89	69.45	53.05	67.33	49.47	43.84
SEDD Absorb	99.59	38.55	72.51	52.16	72.62	47.07	41.18
BD3-LM (L' = 16)	90.63	33.14	64.88	53.09	62.5	43.25	39.82
Eso-LM (B) (Ours)							
α0 = 1	126.29	45.08	82.01	61.37	98.22	62.37	55.76
α0 = 0.5	110.70	39.57	75.75	57.33	86.65	60.20	53.78
α0 = 0.25	105.19	37.32	67.69	60.15	75.74	62.45	55.31
α0 = 0.125	97.46	35.65	60.11	69.13	65.26	65.27	57.4

C.4 OWT에서 훈련된 모델에 의해 생성된 샘플의 생성 퍼플렉시티 및 엔트로피

그림 3에서는 NFEs를 변화시킴에 따라 샘플 품질이 어떻게 변하는지 연구합니다. 생성 PPL 및 엔트로피의 수치는 표 8 (Eso-LMs), 표 9 (MDLM), 표 10 (BD3-LMs)에서 찾을 수 있습니다.

표 8: OWT에서 250K 단계로 훈련된 Eso-LM (B) 샘플의 생성 PPL (↓) 및 엔트로피 (↑).

α0train	**α0eval**	**T**	**NFE**	**Gen. PPL (↓)**	**Entropy (↑)**
0.125	0.0625	16	977.18	23.15	5.37
0.125	0.0625	128	1011.37	25.51	5.41
0.125	0.0625	1024	1022.13	26.85	5.43
0.125	0.25	16	806.47	46.68	5.45
0.125	0.25	128	896.41	44.19	5.42
0.125	0.25	1024	999.24	49.55	5.47
0.125	1.0	16	16.0	279.63	5.54
0.125	1.0	128	127.96	151.77	5.4
0.125	1.0	1024	647.06	159.31	5.42
0.25	0.0625	16	977.18	23.95	5.36
0.25	0.0625	128	1011.37	26.13	5.39
0.25	0.0625	1024	1022.13	27.96	5.41
0.25	0.25	16	806.47	42.31	5.43
0.25	0.25	128	896.41	38.48	5.39
0.25	0.25	1024	999.24	43.71	5.44
0.25	1.0	16	16.0	152.16	5.55
0.25	1.0	128	127.96	72.74	5.4
0.25	1.0	1024	647.06	72.36	5.42
1.0	0.0625	16	977.18	31.33	5.21
1.0	0.0625	128	1011.37	30.9	5.19
1.0	0.0625	1024	1022.13	31.11	5.19
1.0	0.25	16	806.47	50.14	5.4
1.0	0.25	128	896.41	43.59	5.33
1.0	0.25	1024	999.24	43.6	5.32
1.0	1.0	16	16.0	118.92	5.53
1.0	1.0	128	127.96	54.36	5.37
1.0	1.0	1024	647.06	49.4	5.33

표 9: OWT에서 1M 단계로 훈련된 MDLM 샘플의 생성 PPL (↓) 및 엔트로피 (↑).

T	NFE	Gen. PPL (↓)	Entropy (↑)
16	16	148.08	5.61
128	127.96	48.66	5.43
256	251.35	45.07	5.4
512	442.87	42.58	5.37
1024	647.91	41.92	5.36
4096	906.32	41.53	5.36

표 10: OWT에서 1M 단계로 훈련된 BD3-LM 샘플의 생성 PPL (↓) 및 엔트로피 (↑).

Block size	T	NFE	Gen. PPL (↓)	Entropy (↑)
4	4096	925.32	41.04	5.31
4	2048	830.93	88.4	5.19
4	1024	672.91	199.99	4.79
4	512	480.21	186.05	4.23
4	256	256.0	199.22	4.39
8	4096	915.14	39.88	5.33
8	2048	814.07	59.84	5.34
8	1024	646.69	137.99	5.24
8	512	439.32	258.95	4.92
8	256	254.9	278.57	4.39
8	128	128	466.1	4.91
16	4096	910.37	38.74	5.33
16	2048	811.01	47.02	5.34
16	1024	642.25	73.84	5.36
16	512	427.46	189.26	5.33
16	256	245.77	351.63	5.1
16	128	128	447.24	4.61